A Renewable Energy Community Virtual Simulator with a Physics-Grounded Coding-Agent Assistant: service-aware multi-agent control, failure injection, and agent-assisted scenario authoring under grid-compliance verification

Wong Wah Cheung
The Hong Kong Polytechnic University
09900645r@connect.polyu.hk

OpenBot

November 11, 2026

Abstract

We present a renewable energy community virtual simulator whose authoring workflow is assisted by a coding agent: an engineer describes a community in natural language, the agent produces an executable simulation configuration, the engineer confirms it field-by-field before anything runs, and the simulator then evaluates the control strategies of record — rule-based, MPC, single-agent RL, and multi-agent RL, plus the centralized heuristic — across five scenarios (A–E) covering coordination baselines, battery/PV assets, deferrable loads, energy sharing with community settlement, and mid-run component failures. The simulator couples CityLearn building-level simulation with pandapower grid validation across three pricing regimes — pool TOU, locational marginal prices (LMP), and bilateral P2P — at ten seeds each (750 runs), with a further 150-run scenario-D re-run after an energy-sharing configuration defect was fixed, a 150-cell real-LLM screen (100 LLM-arm and 50 reference cells), and a 60-cell hostile-proposer study in two 30-cell pilots; every run carries a manifest recording its full provenance (Section 4.10). The multi-agent controller is cost- and peak-minimal in every scenario (paired cost delta vs. rule-based -11.95 to -43.03 USD across A/B/C/E on the grid-only cost basis, -78.56 USD in scenario D on the settlement-inclusive basis, $p_{\text{Holm}} < 0.0001$, n=10) with an explicit EV-completion trade-off (0.057 vs. 1.000 for MPC) that we report rather than resolve; scenario D rows are settlement-inclusive while A/B/C/E are grid-only, and all cost comparisons are within-scenario. Only price-aware controllers (MPC, RL) change dispatch when the pricing regime changes; a fixed trajectory still bills differently under different price series, so dispatch identity never implies cost identity. Under P2P clearing the A/B/C/E community aggregates are bit-identical to pool clearing while member bills redistribute significantly, and scenario D — whose settlement is already active — diverges at the aggregate as well, through the real grid-cost change from energy sharing. Thermal comfort is unmeasurable on the stock dataset (the indoor-temperature series is entirely NaN), so no comfort claim is made. On the agent layer, the real-LLM dispatch screen is a safety-metrics null (unrepaired violations in 1 of 50 unverified runs, 0 of 49 verified runs; repair iterations zero in both arms; gate: 10 pp), and the hostile-proposer study shows the verified arm accepting zero out-of-bounds proposals in every kept cell against 9.2–13.0 per step unverified, with corrective re-prompt recovering 0.909–1.000 of corrupted proposals (mean 0.970) at a reported cost of recovery. The agent's scenario-authoring front door is held to an explicit acceptance standard rather than trust: a 16-item natural-language corpus with five criteria (complete-config planning, required fields, in-range values, clean simulation, API round-trip), scored on one clean full-16 batch on the final code — 15 of 16 items pass the full bar, 4 of 4 rejection-path items behave correctly, 7 of 8 domains covered; the single non-passing item is an environment data gap (no Singapore weather file ships with the CityLearn datasets, and the field is inert in the code), reported as a limitation rather than

"fixed" by inventing a filename. The agent's autonomous strategy-modification component is measured, not assumed: a two-arm benchmark of the five-agent repair loop repairs 5 of 9 seeded controller defects against a pre-registered bar of 7 of 9 (deterministic comparator 0 of 9, loop and driver verdicts agreeing 10 of 10), a component result we report at its measured level rather than as a platform claim; the repair component's earlier "scaffolded" status is superseded by this sub-bar measurement. All artifacts are reproducible: manifests throughout, bit-identical same-seed repeats, and a measured 0.539 median CI half-width ratio from five to ten seeds.

Keywords: renewable energy communities, virtual simulator, LLM-assisted scenario authoring, multi-agent control, power system simulation, pricing regimes, verification

1 Introduction

Renewable energy community studies presuppose an executable scenario: a device mix, a tariff, a grid model, a weather file. Building that scenario is expert busywork, and it is the part practitioners cannot check. We present a virtual simulator whose authoring step is itself assisted by a coding agent and gated by field-level confirmation, so the scenario an experiment runs is the scenario the engineer approved; the agent is the engineering assistant inside the workflow, not a fifth control strategy, and its code never enters the strategy comparison. Renewable energy communities (RECs) then require coordinated control of distributed energy resources (DERs) under physical grid constraints, service-level commitments, and a market mechanism that prices energy, and existing approaches typically optimize a single objective (cost, peak, or carbon) within one fixed setting — one tariff, one device mix, one assumption that controllers execute what they compute. Four research questions follow, each established by a distinct literature, and this paper answers the parts of them our experiments actually support.

(0) Scenario authoring acceptance. A scenario configuration is an artifact with required structure: buildings with devices, tariffs, a grid model, a weather file, and a simulation window, all in range. The research question is whether an agent-authored configuration can be held to an acceptance standard that names its own failures rather than silently auto-correcting them. Our answer is an instrument and a measurement: a 16-item natural-language corpus with five criteria (complete-config planning, required fields, in-range values, clean simulation, API round-trip), including deliberately incomplete and out-of-range descriptions whose rejection must name the offending field. The authoring acceptance number is reported in Section 4.1 at its measured level, and the paper claims no authoring number the corpus has not produced.

(1) Pricing-regime diversity. Community-control benchmarks and datasets fix a single price-formation mechanism: the CityLearn challenge series ships time-of-use tariffs; ElecBench scores power-dispatch knowledge under fixed assumptions Zhou et al. [2025]; PowerAgentBench-SS scores tool-using agents on steady-state studies Mylonas et al. [2026]; distributed MPC and dual-decomposition formulations Yfantis et al. [2024], Su et al. [2021] take the price signal as given. The unresolved question is whether a controller's *dispatch* and its *billed cost* respond differently when the mechanism changes — comparisons that price one fixed trajectory under two series cannot separate the two responses, and dispatch identity never implies cost identity. Our finding, over a 750-run matrix (5 scenarios × 5 controllers × 10 seeds × 3 regimes) plus a 150-run scenario-D re-run, is the separation itself: only the price-aware controllers (MPC, RL) change dispatch across regimes, the price-insensitive heuristics dispatch identically, the LMP price series reaches the bills only where settlement and dynamic pricing are active (scenario D), and bilateral P2P clearing redistributes member bills while the A/B/C/E community aggregate is bit-identical to pool clearing. The contribution is the dispatch/billing separation and the mechanism account of which regime reaches the bills, not a TOU-versus-LMP cost comparison.

(2) Failure injection. Injection itself is standard practice in power-systems and building-simulation benchmarks and measures the scenario, not the agent. The research question is

the four-step chain a controller must complete: identify the failure type, localise the affected controller, produce an effective repair, and maintain grid constraints and participant service after the repair; no benchmark in this literature reports the chain. We measure the chain and state our position on each step. Scenario E injects PV inverter, battery, EV-charger and sensor failures at severity 1.0 plus a grid-service limit reduction, in 150 runs across all three regime legs. Identification at the grid level is *empty* — power flow converges in every step of all 150 runs with zero voltage, line and transformer violations and zero re-dispatches, so no controller observes the failures and detection is configuration-oracle only. Localisation is oracle-side only, by construction of the injection. Repair is *measured and sub-bar*: the commissioned code-repair benchmark repairs 5 of 9 seeded controller defects with the agent arm against a pre-registered bar of 7 of 9 (deterministic comparator 0 of 9; 44 backend calls, zero backend errors, loop and driver verdicts agreeing 10 of 10), so the repair workflow is demonstrated at a measured level below its bar rather than left unmeasured. Maintaining constraints and service is trivially satisfied because no injected failure produced a violation in these runs. We claim no detection and no recovery for any failure type.

(3) Verified LLM code agents. This is the paper's contribution. Control code is executable, so an emitted "change" that does not change behaviour is not a repair: without an executable check the failure mode is silent. Our own instrumented coder produced comment-level patches until an AST-transformation layer was restored, and the ten-task code-repair benchmark measured zero of nine measured repairs with a hosted model in the loop, because the model emitted change descriptions while the edit was applied by a fixed transformation vocabulary. Adjacent verification layers exist in the literature — solver-grounded reporting Rojas et al. [2026], intent-driven separation of planning from solver-side execution Mao et al. [2026], and runtime interception of proposed actions against derived specifications Batole et al. [2026] — but none measures whether agent-generated *code* changes behaviour, and none prices the verification layer against a deterministic alternative. We therefore separate the loop into five specialised agents rather than one agent plus a deterministic verifier: the Planner diagnoses the violation set and targets functions, the Coder generates the change, the Executor runs the patched controller in a sandbox with the real test suite, the Verifier checks violation resolution against held-harmless service criteria, and the Reporter emits a machine-readable per-iteration audit; the decomposition puts a deterministic checkpoint at every boundary and makes each stage independently auditable. The workflow's value is then measurable, and we measure it: the verification machinery is a no-op under an in-bounds proposer (a safety-metrics null, reported as such) and engages fully under a deliberately out-of-bounds proposer — the verified arm accepted zero out-of-bounds proposals in every kept cell, 13/13 paired directional wins, exact sign test $p = 2.4 \times 10^{-4}$ (reference-only). That is runtime proposal verification measured against ground truth, which is what this paper contributes.

Related work spans four areas. For building- and home-scale agentic energy management, tool-calling ReAct agents convert natural-language requests into appliance schedules over dynamic tariffs and PV forecasts Jonah et al. [2026]; hierarchical agentic HEMS put the LLM in the decision loop El Makroum et al. [2026]; and multi-agent assistants add conversational interfaces with domain-specific tool registries Jung [2026]. A recent synthesis of smart-grid LLM agents states the solver-grounded principle we instantiate — a numerical result is reported only when it comes from a trusted tool and passes verification Rojas et al. [2026]. For verification of agent actions, intent-driven workflows separate LLM planning from solver-side execution and gate results on state-dependent constraint checks Mao et al. [2026], and runtime verification intercepts proposed actions against derived specifications Batole et al. [2026]. For benchmarking, ElecBench scores power-dispatch knowledge Zhou et al. [2025] and PowerAgentBench-SS scores tool-using agents on steady-state studies with evidence logs and risk-sensitive metrics Mylonas et al. [2026]; neither covers failure injection or settlement, which this matrix adds. Multi-agent systems for smart grids are surveyed in Izmirlioglu et al. [2024]. Finally, dual decomposition

for distributed MPC, including asynchrony and inexact subproblem solves, is benchmarked and analysed in Yfantis et al. [2024], Su et al. [2021]. Simulation and validation build on CityLearn Vázquez-Canteli et al. [2020] and pandapower Thurner et al. [2018], with model-predictive control Rawlings et al. [2020], tabular RL Zhou et al. [2022], AST-based patching Rall et al. [2020], and the Holm correction for multiple comparisons Holm [1979].

2 System and Method

2.1 Simulation Coupling

CityLearn handles building-level DER dynamics (battery SOC, EV charging, flexible loads, thermal comfort) Vázquez-Canteli et al. [2020]. pandapower validates grid physics (power flow convergence, voltage limits ±5%, line/transformer thermal limits) Thurner et al. [2018]. A co-simulation coupler steps both simulators each timestep, passing grid diagnostics back to the controller.

2.2 Scenario-Authoring Pipeline

The coding agent converts a natural-language description into an executable `ScenarioConfig` through a plan–code–execute–verify–report loop: a planner extracts the target assets and per-building device parameters, a coder writes the configuration, an executor runs a quick simulation, and a verifier checks schema validity, serialization round-trip, and determinism before the engineer confirms the configuration field-by-field. A rule-based keyword planner is the fallback only when the LLM backend is unavailable, and the fallback is recorded per item in the manifest's planner provenance rather than silently substituted.

2.3 Controller Suite

`rule_based` Heuristic priority: charge battery from excess PV, charge EVs by departure deadline, shift flexible loads to solar hours, enforce community import/export limits.

`multi_agent` Decentralized per-building agents with consensus on community-level limits via dual decomposition.

`centralized` Heuristic joint dispatch with full observability (not an optimizer).

`mpc` Receding-horizon LP with perfect 24h forecasts (upper bound on service) Rawlings et al. [2020].

`rl` Tabular Q-learning per scenario (126-state base encoder; 138-state for LMP) Zhou et al. [2022].

All controllers share one action interface (requested kW per configured asset) and one clamp-repair path: whatever a controller requests is normalized to the device action range before the simulator applies it, so both LLM arms execute clamped values. The verified/unverified difference is entirely in the proposal-layer check (Section 4.7), not in post-clamp execution.

2.4 Verified LLM Dispatch Verification and the Code-Repair Loop

A five-agent loop iteratively repairs controller code:

1. **Planner**: Diagnoses violations, identifies root cause, proposes repair strategy.
2. **Coder**: Applies AST-based patches to target functions Rall et al. [2020].

3. **Executor**: Runs patched controller in sandbox with unit tests + quick simulation.

4. **Verifier**: Counts violation resolution from sandbox diagnostics.

5. **Reporter**: Generates comprehensive report with diffs and recommendations.

The loop runs for max 3 iterations or until `violations_resolved=true`. **Scope statement:** the loop's strategy-modification component is measured at its benchmark level, not assumed: a two-arm benchmark against nine seeded controller defects repairs 5 of 9 defects with the agent arm against a pre-registered bar of 7 of 9 (deterministic comparator 0 of 9; loop and driver verdicts agreeing 10 of 10, both arms unpooled). The sub-bar result is reported rather than resolved; what the paper additionally measures is the runtime verification layer that guards LLM *dispatch proposals* at every step (Sections 4.7 and 4.8).

The ablation compares `llm_agent` (single-pass, execute-as-proposed) vs. `llm_agent_verified` (per-step verification, re-prompt once, fallback to rule-based on failure, repair cap 2). Both run in `fallback` mode (deterministic rule-based proposer) to isolate verification machinery.

The real-LLM screen (M2a) uses a hosted OpenAI-compatible endpoint (temperature 0, 120 s per-call timeout, prompts `llm_dispatch_v2` / `llm_dispatch_v1_repair`); model identity and provider are withheld per operator policy, with full provenance in the Reproducibility section. The 150-cell screen comprises 100 LLM-arm cells (5 scenarios × 2 LLM arms × 10 seeds) plus 50 `multi_agent` 24-step reference cells (149 cells landed; one verified cell quarantined), and is described in Section 4.7.

3 Experimental Setup

Dataset and horizons. All runs use the stock CityLearn 2022 challenge dataset (`citylearn_challenge_2022` five buildings, hourly resolution, one simulated week available). Scenarios declare 15-minute steps but the adapter aligns them to the dataset's native 60-minute resolution; every reported run uses a 24-step (one-day) horizon. Load and PV traces depend only on (seed, filename) — a seed selects CityLearn's generated fallback data, and the same seed reproduces the same traces for every controller — so controller differences at equal seed are pure controller effects, and seed variance reflects data generation, not controller nondeterminism. What changes with the scenario is the device mix and the community/sharing configuration; what changes with the controller is only the policy.

Five scenarios (Table 1) differ in device mix and stressors. Each runs ten seeds (42, 123, 456, 789, 999, 1234, 2024, 3456, 5555, 8888) × five controllers = 250 runs per regime, under three regimes (pool TOU, LMP, bilateral P2P) for 750 runs; the legs of record are `matrix_full` (pool TOU, A/B/C/E) plus `matrix_d_rerun` (D), `m52_benchmark` (LMP, A/B/C/E; the directory name is a legacy of its M5.2 origin) plus `matrix_d_rerun_lmp` (D), and `matrix_p2p` (P2P, A/B/C/E) plus `matrix_d_rerun_p2p` (D). An energy-sharing configuration defect was found and fixed; the settlement fix is not in the `matrix_full` or `m52_benchmark` lineages, so their D blocks are pre-fix and superseded, and scenario D's rows are cited from the 150-run re-run (5 controllers × 10 seeds × 3 regimes) in all three regimes. Postfix copies and the `matrix_d_rerun` pool-TOU block duplicate D rows and are never added on top of the 750. D's rows are settlement-inclusive, A/B/C/E report grid-only cost. A further 50-run fallback-mode ablation and a 150-cell real-LLM screen (100 LLM-arm and 50 reference cells, 24 simulated steps) isolate verification machinery from proposer quality, and a 60-cell hostile-proposer study in two 30-cell pilots forces verifier engagement. All cost comparisons in this paper are therefore within-scenario.

Cost basis. Only scenario D enables community energy sharing, so its `total_cost` is the community bill including settlement (190.81 ± 0.03 USD for `rule_based`), while A/B/C/E report the grid-only bill. Cross-scenario cost magnitudes are not comparable; every comparison we make is a within-scenario paired delta, which is unaffected.

Comfort is unmeasurable here. The stock CityLearn 2022 dataset ships the indoor dry-bulb temperature series entirely NaN, so `comfort_violation_degree_hours` is 0.0 in every run of this study and measures data availability, not comfort satisfaction. We make no comfort claim; EV completion is reported in full (Section 4.9).

Table 1: Scenario definitions. "No added DERs" means the stock dataset device mix (battery, PV, EV chargers and deferrable loads as shipped); scenario-added assets are listed per row.

Scenario	Device Mix	Stressor
A	Base, no added DERs (stock EVs included)	Coordination baseline
B	+ Battery, PV	Tariff exposure
C	+ Flexible loads	Deferrable load deadlines
D	D as C + energy sharing	Community settlement (cost basis: settlement-inclu
E	Full + Failures	Mid-run component failures

4 Results

4.1 Scenario Authoring Acceptance (Core 3)

The coding agent's front door is natural language: an engineer describes the community and the agent produces an executable `ScenarioConfig`, which the engineer confirms field-by-field before anything runs. Acceptance is not trust: a 16-item natural-language corpus (twelve complete descriptions covering buildings, PV, battery, EV, flexible loads, tariff, weather and grid, plus two deliberately incomplete and two out-of-range descriptions) is scored under five criteria — the plan is a complete configuration, required fields are present, values are in range, the scenario simulates clean, and the config round-trips through `POST /api/scenarios` and `GET /api/scenarios/{id}/validate`. Rejection-path items invert the first three: an invalid or incomplete description must not be silently auto-corrected, and the offending field must be named. The pre-registered bar is at least 14 of 16 items, all eight domains covered, and all four rejection items behaving correctly.

Result (one clean full-16 batch on the final code, scored with the benchmark's own instruments, no stitched numbers): 15 of 16 items pass the full bar, 4 of 4 rejection items behave correctly, and 7 of 8 domains are covered. The single non-passing item is a data gap, not a pipeline defect: no Singapore weather file ships with the CityLearn datasets in this environment (each dataset carries one weather.csv) and no simulator code reads `BuildingConfig.weather_file`, so the canonical weather token is the environment's honest ceiling. We report it as a limitation in the Limitations section rather than "fix" it by inventing a filename. Every item's plan is agent-produced (planner provenance recorded per item as `llm`, with the rule-based keyword fallback never silently substituted). The measurements, the corpus, and the scorer are released with the paper's artifacts.

4.2 Cost, Peak and Service

Figure 1 shows KPI bars with 95% CI. The cost-versus-service Pareto front (Figure 2) places `multi_agent` at the cost-minimal end and `mpc` at the service-maximal end in every scenario. Table 3 reports paired per-seed deltas vs. `rule_based` with Holm correction.

Cost is reported at achieved service, and sub-threshold service is marked infeasible. The controllers do not share an objective or a service guarantee (Table 2): `multi_agent` and `rule_based` are heuristics with no EV-deadline constraint, `centralized` is a full-observability heuristic joint dispatch, and only `mpc` and `rl` carry an EV service term. In scenario A,

`multi_agent` meets 0.057 of departure requirements against `mpc`'s 1.000; its cost advantage is therefore a cost advantage at a lower service level, and we mark controllers below a full-EV-completion threshold as service-infeasible rather than cost-advantaged. No equal-service cost comparison is claimed; closing the EV gap under cost parity is open work.

Table 2: Controller objectives, constraints, and observations (source of truth: `experiments/BENCHMARK_MATRIX.md`). Service feasibility is the EV departure-SOC requirement; only MPC and RL carry it.

Controller	Objective	Constraints enforced	Observations
rule_based	heuristic priority (PV excess, EV deadline, load shifting)	community import/export limits; device bounds (clamped)	full community observations
multi_agent	per-building heuristic + consensus on community limits	community import/export limits; device bounds (clamped)	per-building observations
centralized	heuristic joint dispatch (not an optimizer)	community import/export limits; device bounds (clamped)	full community observations
mpc	24-h LP cost with peak proxy and EV shortfall penalty	device power/SOC limits; EV deadline term	causal 24-h forecasts (perfect in this build)
rl	tabular Q-learning on SOC/price/time state	device bounds via action discretization (clamped)	discretized SOC/price/time features

The service trade-off, stated explicitly. `multi_agent` is cost-minimal (e.g. 22.39 ± 0.00 vs. rule-based 40.49 ± 0.03 USD in scenario A) but completes only 0.057 of EV departure requirements, against 0.154 for rule-based and 1.000 for MPC. Under the paper's service-aware objective this is a defect, not a win: the controller buys cost and peak reductions (-16.59 kW) by under-charging EVs. We therefore report cost, peak, and EV completion as jointly necessary, and treat closing the EV gap under cost parity as open work rather than an achieved property. With restored PV, several scenario C configurations are net exporters and their total cost is negative (export credit exceeding import cost); negative totals are flagged as export-credit, not as profit.

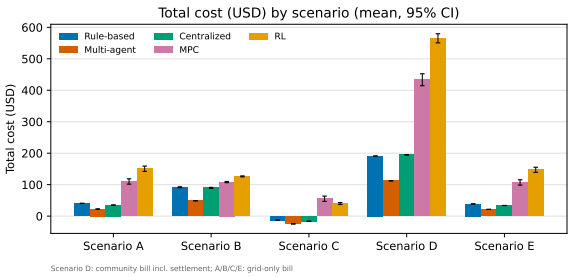
Scenario D on the re-run basis. Scenario D's rows use the re-run basis, on which `total_cost` is the settlement engine's actual community bill when sharing is enabled; D diverges from A at all ten seeds on that basis. The physics — grid exchange, PV, peak — is unchanged between the two bases; only the cost basis moved, from grid-only to community-bill-including-settlement. A per-run accounting invariant (internal P2P payments sum to zero; community aggregate cost equals the external grid bill plus explicit community trades) is asserted by regression tests, so D's aggregate movement is a property of the settlement-inclusive basis, not an accounting defect.

Table 3: Paired per-seed deltas vs. `rule_based` (mean ± 95% CI half-width; n=10 seeds; Holm within each scenario over the four controller comparisons). Scenario D is settlement-inclusive and A/B/C/E grid-only; comparisons are within-scenario (Section 3). Cost deltas must be read together with service feasibility (Table 2, Section 4.9).

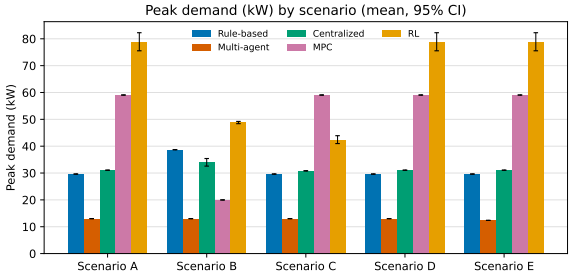
Scenario	Controller	Δ cost (USD)	Δ peak (kW)	p_{Holm}
A	multi_agent	-18.11 ± 0.03	-16.59 ± 0.00	< 0.0001
A	centralized	-5.26 ± 0.22	$+1.43 \pm 0.00$	< 0.0001
A	mpc	$+69.60 \pm 8.54$	$+29.44 \pm 0.00$	< 0.0001
A	rl	$+110.09 \pm 8.41$	$+49.28 \pm 3.37$	< 0.0001
B	multi_agent	-43.03 ± 1.45	-25.68 ± 0.00	< 0.0001
B	centralized	-2.06 ± 0.62	-4.71 ± 1.41	< 0.0001
B	mpc	$+16.35 \pm 2.66$	-18.73 ± 0.00	< 0.0001
B	rl	$+34.41 \pm 2.14$	$+10.10 \pm 0.42$	< 0.0001
C	multi_agent	-11.95 ± 0.04	-16.59 ± 0.00	< 0.0001
C	centralized	-3.68 ± 0.16	$+1.20 \pm 0.00$	< 0.0001
C	mpc	$+68.03 \pm 8.33$	$+29.44 \pm 0.00$	< 0.0001
C	rl	$+52.69 \pm 3.06$	$+12.83 \pm 1.47$	< 0.0001
D	multi_agent	-78.56 ± 0.03	-16.59 ± 0.00	< 0.0001
D	centralized	$+3.99 \pm 0.24$	$+1.43 \pm 0.00$	< 0.0001
D	mpc	$+242.63 \pm 18.93$	$+29.44 \pm 0.00$	< 0.0001
D	rl	$+374.34 \pm 14.41$	$+49.28 \pm 3.37$	< 0.0001
E	multi_agent	-17.46 ± 0.03	-17.26 ± 0.00	< 0.0001
E	centralized	-5.06 ± 0.22	$+1.43 \pm 0.00$	< 0.0001
E	mpc	$+68.18 \pm 8.48$	$+29.44 \pm 0.00$	< 0.0001
E	rl	$+108.45 \pm 8.09$	$+49.28 \pm 3.37$	< 0.0001

Table 4: Scenario D KPI comparison (pool TOU, re-run basis, n=10). Total cost is the community bill including settlement.

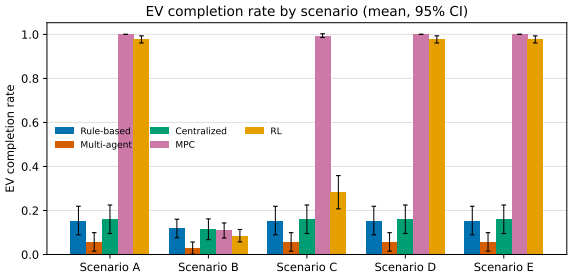
Controller	Total cost (USD)	Peak demand (kW)	EV completion
rule_based	190.81 ± 0.03	29.62 ± 0.00	0.154 ± 0.062
multi_agent	112.26 ± 0.00	13.03 ± 0.00	0.057 ± 0.039
centralized	194.80 ± 0.25	31.05 ± 0.00	0.160 ± 0.065
mpc	433.44 ± 18.93	59.06 ± 0.00	1.000 ± 0.000
rl	565.15 ± 14.41	78.90 ± 3.37	0.977 ± 0.021



(a) Total cost (USD)



(b) Peak demand (kW)



(c) EV completion rate

Figure 1: KPI comparison bars with 95% CI whiskers per scenario.

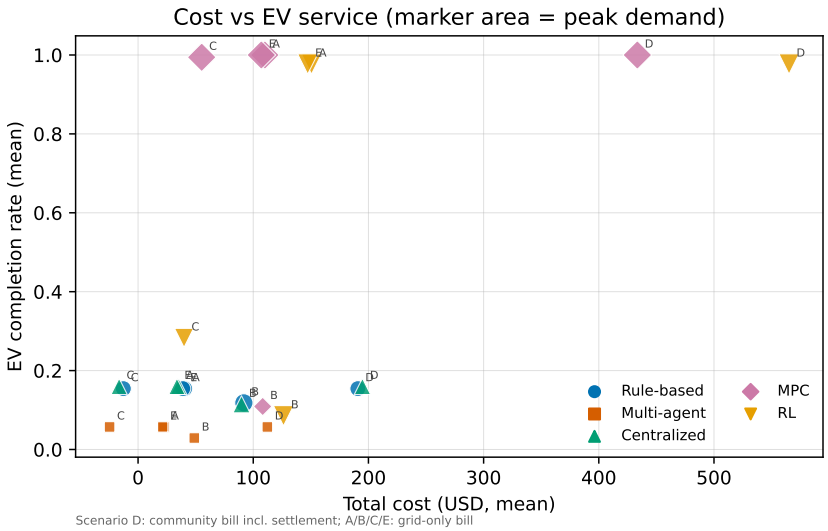


Figure 2: Cost vs. EV completion Pareto scatter (marker area = peak demand).

4.3 LMP-Regime Benchmark

250 runs under `lmp` pricing (5 controllers × 5 scenarios × 10 seeds; scenario D on the re-run, settlement-inclusive basis). The central finding is a *price-awareness split*: only the two price-aware controllers change dispatch when the regime changes. MPC's cost rises from 110.09±8.54 USD (TOU) to 168.47 ± 10.69 (LMP) in scenario A and its EV completion drops (1.000 → 0.966), and RL's arm shifts similarly (150.58 → 106.22 in scenario A), while rule-based, multi-agent, and centralized — none of which condition on prices — produce identical dispatch under both regimes (e.g. multi_agent 22.39 ± 0.00 in both). Ranking within a regime is unchanged: `multi_agent` cost-minimal (all $p_{\text{Holm}} < 0.0001$), `centralized` close to `rule_based`.

Dispatch sensitivity is not billing sensitivity. Cost is a function of prices on a fixed trajectory, so identical dispatch across regimes does not imply identical cost. We separate the two: dispatch sensitivity is the per-step action comparison above (identical for the price-insensitive controllers, different for MPC/RL); billing sensitivity is a property of the billed series. A billing-identity test prices one fixed trajectory under the TOU and LMP series and asserts the bills differ (84.43 vs. 79.04 USD in the test configuration), and a companion test asserts the LMP series enters billing for every building. In the landed matrix, A/B/C/E costs are unchanged across all three regimes for the price-insensitive controllers (200 paired cells, zero differing): those scenarios run without community settlement, so the billed series does not change with the regime — a property of the billing configuration in these builds, not a consequence of dispatch identity. Scenario D, where settlement and dynamic pricing are active, does bill the LMP series, and its regime rows are cited from the dedicated re-run matrices (all three matrix legs — pool TOU, LMP, and P2P — still carry D's pre-re-run rows in their D blocks): e.g. rule_based 190.81 USD (TOU) vs. 165.03 (LMP) on the settlement-inclusive basis.

LMP construction, stated as built. "LMP" here is not a network-dual locational marginal price from the co-simulation power flow. In our builds the community network runs a separate pandapower OPF pass each step (slack bus and storage dispatchable, PV treated as fixed injection); the per-bus power-balance duals of that OPF are converted to $/kWh, capped, and fed into the settlement engine as a price series (a voltage-proportional approximation is the OPF fallback). It is therefore a price series constructed from a network OPF, and the name is used in that sense throughout.

Table 5: Scenario D under LMP (re-run basis, n=10; settlement-inclusive). Rows cited from the dedicated scenario-D re-run matrices (`matrix_d_rerun`, `matrix_d_rerun_lmp`).

Controller	Cost TOU (USD)	Cost LMP (USD)	Δ	EV TOU	EV LMP
rule_based	190.81 ± 0.03	165.03 ± 0.00	−25.79	0.154	0.154
multi_agent	112.26 ± 0.00	94.32 ± 0.00	−17.94	0.057	0.057
centralized	194.80 ± 0.25	173.31 ± 0.00	−21.49	0.160	0.160
mpc	433.44 ± 18.93	424.17 ± 9.03	−9.27	1.000	0.971
rl	565.15 ± 14.41	310.29 ± 8.23	−254.86	0.977	0.411

4.4 P2P Regime: Bilateral Clearing Redistributes, Aggregates Unchanged (A/B/C/E)

250 runs under `p2p` pricing (bilateral double-auction clearing with uniform residual price, same grid as Section 3). Paired against the pool-TOU matrix at identical (scenario, controller, seed), the A/B/C/E community aggregates are *bit-identical*: Δ cost, Δ peak, and Δ EV completion are exactly zero across the 200 paired runs (4 scenarios × 5 controllers × 10 seeds; no controller excluded), while bilateral trades clear in every run (mean community volume 347.2 kWh, max

563.1, no zero-volume runs). This cost-transparency is by construction: trades reallocate energy between members before power flow, leaving the residual grid exchange unchanged.

The effect is at the member level. Per-member bills shift significantly under P2P (paired, n=40 runs per member): building_2 pays 7.60 ± 1.83 USD less, while building_1 (-5.64 ± 1.97), building_3 (-7.17 ± 1.16), and building_4 (-2.37 ± 2.04) pay more; building_0's change is not significant ($+1.92 \pm 4.60$). Aggregate savings vs. baseline are statistically indistinguishable from pool clearing (-5.66 ± 10.09 USD, paired p=0.26). The fairness coefficient as computed is invariant (0.0926 in both arms, zero paired variance), which exposes a measurement gap: the current Gini input does not reflect settlement redistribution. We therefore report RQ3 as *redistribution without aggregate benefit* under the tested regime for A/B/C/E, and defer inequality claims until a settlement-aware fairness metric is defined.

Scenario D is the exception. Because community settlement is already active in D, bilateral clearing adds a second redistribution layer on top of it and the aggregate moves as well (paired, n=10; rows cited from the dedicated scenario-D re-run matrices): rule_based $+9.58 \pm 0.91$ USD, multi_agent -24.10 ± 0.00, centralized $+12.08 \pm 1.04$, mpc $+29.06 \pm 4.63$, rl $+34.02 \pm 0.91$, with peak and EV completion exactly unchanged and community volume 252–520 kWh. The "aggregates bit-identical" claim is therefore scoped to A/B/C/E; D is genuinely divergent under P2P on the settlement-inclusive basis. The mechanism is not double counting: the second settlement layer redistributes member bills (internal payments sum to zero, asserted by a regression test), and the aggregate moves only through the real grid-cost change from energy sharing — the community aggregate equals the external grid bill plus explicit community trades under the accounting invariant.

4.5 Scenario E: Failure Injection

Scenario E is the paper's aggressive failure configuration: it injects PV inverter, battery, EV charger, and sensor failures at severity 1.0 mid-run (the EV charger is completely unavailable during its window) plus a temporary grid-service limit reduction to 95% of the community import limit; every configured failure activates and deactivates on schedule. This configuration is identical in all three pricing-regime legs, so the aggressive parameters are a scenario-level asymmetry (A/B/C/D use the standard configuration in every leg), not a regime-specific one. The matrix covers 150 runs (5 controllers $\times$ 10 seeds under each of the three regimes; `rl` included — 10 runs per controller per regime, 30 per controller across regimes). All 150 runs show `members_interrupted = [building_3]` and `member_cost_coverage = 0.80`: the sensor-failure window blanks one member's cost data, which is reported as an interruption, never scored as zero cost.

At the grid level, none of the injected failures is detectable from the simulation in these runs: power flow converges in every step of all 150 runs, voltage and loading stay inside limits (zero voltage/line/transformer violations), and no re-dispatch is ever triggered — for `rl` exactly as for the four other controllers. Detection of the failures is therefore configuration-oracle only — no controller observes them — and we make no detection or recovery claim for any failure type, the aggressive grid-service limit reduction included.

4.6 Ablation: Verified vs. Unverified Agents

All 50 fallback-mode ablation runs (`agent_backend: fallback`) use a deterministic rule-based proposer. Under it both LLM arms are KPI-identical to `rule_based` and the verification machinery never fires: first-attempt validity 1.000, repairs 0.00/step, verification failures 0.00, unparseable outputs 0.00, wall time 16–33 s per run (median 20.8 s), no arm separation. The null result has two causes: the fallback proposer emits in-bounds proposals, so the verification loop never triggers; and both arms share one clamp-repair path downstream, so execution-level behaviour is identical by construction. The verified arm's difference is at the proposal layer:

`llm_agent` applies proposals as proposed, while `llm_agent_verified` re-prompts once, then falls back. Sections 4.7 and 4.8 report what happens when a real — and then a deliberately hostile — proposer is substituted: a safety-metrics null, and then forced verifier engagement.

4.7 Real-LLM Screen: M2a (Proposal-Layer Verification, Safety-Metrics Null)

The M2a screen replaces the deterministic fallback proposer with a hosted LLM endpoint on the same 24-step horizon (temperature 0, 120 s per-call timeout, prompts `llm_dispatch_v2` / `llm_dispatch_v1_repair`; model identity and provider withheld per operator policy, provenance in Section 4.10). The 150-cell screen — 100 LLM-arm cells (5 scenarios × 2 arms × 10 seeds) plus 50 `multi_agent` 24-step reference cells — yielded 99 usable LLM-arm runs (50 unverified, 49 verified; one verified cell quarantined for endpoint degradation), and every LLM-arm manifest records `agent_backend=api`. Because the simulation adapter clamps every action to device limits in both arms, this screen measures verification at the proposal layer, not simulator-level physical safety. Per-arm agent-execution aggregates (run-level event rates unless stated):

Table 6: M2a real-LLM screen, per-arm agent-execution aggregates (n=50 unverified, n=49 verified; mean ± 95% CI half-width).

Metric	`llm_agent`	`llm_agent_verified`
First-attempt validity (per run)	1.0000 ± 0.0000	1.0000 ± 0.0000
First-attempt validity (per step)	0.9967 ± 0.0032	0.9991 ± 0.0017
Repair iterations (per run)	0.0000 ± 0.0000	0.0000 ± 0.0000
Fallback-step share (per run)	0.0025 ± 0.0028	0.0009 ± 0.0017
Fallback steps (per run)	0.0600 ± 0.0682	0.0204 ± 0.0410
Unrepaired violations (per-run rate)	0.0200 ± 0.0402	0.0000 ± 0.0000
LLM calls (per run)	24.0000 ± 0.0000	24.0000 ± 0.0000
Parse failures (per run)	0.0600 ± 0.0682	0.0204 ± 0.0410
Prompt tokens (per run)	29,736 ± 93	29,735 ± 95
Latency p50 / p95 (s)	4.68 / 8.97	4.56 / 8.92

Paired against the `multi_agent` reference cells, the KPI differences are (mean ± CI half-width): unverified $+28.58 \pm 21.07$ USD cost and $+5.42 \pm 4.14$ kW peak; verified $+5.83 \pm 5.92$ USD and $+1.00 \pm 1.14$ kW. The arm difference (unverified − verified) is $+23.05 \pm 21.32$ USD (paired t=2.174, p=0.0347, n=49) and $+4.53 \pm 4.24$ kW (t=2.148, p=0.0368).

Gate verdict — a safety-metrics null. The pre-registered gate is the two percentage-point clauses: proceed to a full-week confirmation only if the arms differ by ≥ 10 points on `llm_unrepaired_violations` or `llm_fallback_step_share`. Both are far below the threshold (0.02 and 0.0017 points; unrepaired violations are nonzero in 1 of 50 unverified runs and 0 of 49 verified runs), so the gate is *not met* and the screen reports a null result on the safety metrics: under this proposer the verification machinery has nothing to catch. The KPI differences above are reported as *context only*: they say the unverified proposer is worse, not that the verifier adds safety value, and no verification-benefit claim is made from them. The validity gate is passed (fallback-share 0.0025 ≪ 0.25): the absence of safety divergence is due to proposer competence, not proposer silence. This null result is specific to the model–prompt–horizon combination, which is exactly what the hostile-proposer study (Section 4.8) was designed to break.

4.8 Hostile-Proposer Study: M2b (Proposal-Layer Verification Under a Hostile Proposer)

Because the M2a proposer emits in-bounds proposals, the verification machinery never engages. The M2b study injects a deterministic corruption at the parsed proposal path (env

flag `SAEA_LLM_HOSTILE=out_of_bounds`), scaling every first-attempt proposal past its device power limit, and runs two 30-cell pilots (5 scenarios × {llm_agent, llm_agent_verified} × 3 seeds 42/123/456, 24 steps, pool TOU, reference cells reused from the M2a screen at the same (scenario, seed) pairs — the 15 deterministic `multi_agent` cells itemised in the Table 7 caption). The first pilot (detection epoch) corrupts every parsed proposal including the verified arm's corrective re-prompt; the second (recovery epoch) exempts re-prompts so the repair path is measurable. Both arms see identical corrupted proposals, so verification is the only difference between them; the adapter clamp downstream is shared by both arms, so all results are proposal-layer verification results, not simulator-level physical-safety results.

Table 7: M2b hostile-proposer study, recomputed from run manifests and `result.json` (provenance: `docs/count_audit/table7_rebuilt.md`; per-cell values: `docs/reviewer_response/e1_cells.csv`; aggregation decision: `docs/reviewer_response/m2b_aggregation_decision.md`). Chosen F7 basis, one cell set per quoted number and never mixed: on-disk manifests, with the keep-13 set (13 cells, 312 steps) carrying both the fallback and the cost rows and the as-run all-15 set (15 cells, 360 steps) quoted for both of its own rows; the archive-substituted construction (18/360) is invalid because the archive holds round-1 degraded copies, not the in-place re-run. Definitions and denominators, stated once: "fallback steps" is the recorded `llm_fallback_steps` counter summed over the cell set (denominator = 24 steps × cells in the set), and "cost" is the mean per-cell `total_cost`. Headline = KEEP cells under the rule `llm_backend_errors` ≤ 2 AND `llm_calls` $\geq$ steps AND telemetry recorded, per epoch and never pooled (detection 15/15 cells; recovery 13/15 cells, denominator 312 steps, after a confirmation re-run of the three quarantined cells was applied in place: C/42 promoted at 0 errors, C/123 and E/42 remain quarantined at 3 endpoint errors and are published per cell, not pooled). The unfiltered as-run sets are published alongside for audit (detection: 360/360 steps, 203.44 USD; recovery: 15/360 steps, 442.11 USD). Repair success in the detection epoch is derived (720 repair iterations minus 360 deterministic replacements; the recorded field is null in that build). Reference = 65.52 USD over exactly 15 cells: the deterministic `multi_agent` 24-step pool-TOU runs at seeds 42/123/456 across scenarios A–E in `experiments/m2a_rerun_laguna` (per-scenario means 66.29/68.81/59.93/66.29/66.29 USD, three seeds each; every manifest records `git_dirty: false`, build commits 9588cb5e4/67d3e276b/95b328d6e). The per-cell list (run id, cost, peak, per-cell `result.json` SHA-256, build commit) is committed at `docs/reviewer_response/m2b_reference_cells.csv`, and the runs ship in the release artifact `dist/m2a-laguna-matrix-2026-11-08.zip` (SHA-256 `660dc7b2...`), so the aggregate recomputes from the committed list or the archived artifact (the 15 cells are bit-identical between the shared checkout and the artifact, SHA-256 verified).

	Detection epoch	Recovery epoch
Cells (verified arm)	15 keep / 0 quarantined	13 keep / 2 quarantined
Unsafe acceptance, unverified (per step)	10.60	10.30
Unsafe acceptance, verified (per step)	0.0000	0.0000
Paired directional wins	15/15	13/13
Bound-violation detections (per cell)	24/24	24/24
Repair success rate (verified)	0.500 (derived)	0.909–1.000 (mean 0.970)
Fallback steps (verified total)	360/360 (all 15 cells)	9/312 (13 keep; 15/360 as-run all-15)
Cost, verified (USD, mean)	203.44 (all 15 cells)	440.85 (13 keep; 442.11 as-run all-15)
Cost, unverified (USD, mean)	440.79	437.15

What the study shows. Under a proposer that is deliberately out-of-bounds, the unverified arm applies the corrupted proposal and the adapter clamps it downstream — an unsafe

acceptance rate of 9.2–13.0 out-of-bounds values per step across cells (mean 10.60 detection, 10.30 recovery over the kept pairs), while the verified arm accepted zero out-of-bounds proposals in every kept cell (15/15 paired directional wins in the detection epoch; 13/13 after a confirmation re-run of the three quarantined cells promoted one cell to keep, two remaining quarantined for endpoint degradation; exact sign test on the 13 pairs, two-sided $p = 2.4 \times 10^{-4}$, reported for reference only). In the recovery epoch, where re-prompts are exempt from corruption, the corrective re-prompt returned an in-bounds proposal that was applied in 0.909–1.000 of attempts (recorded `repair_success_rate`; mean 0.970 over the 13 kept cells; the 0.880 floor belongs to the two quarantined cells), so the loop demonstrably repairs rather than only rejects. The detection epoch — whose re-prompts were themselves corrupted — could not repair by construction (derived repair success 0.500) and ended every attempt on the deterministic replacement dispatch (360/360 steps); we report that epoch as detection-plus-replacement, not recovery, with 15 of 15 verified cells kept.

The cost of recovery. The verified arm's KPIs track the unverified arm rather than the deterministic reference: cost 440.85 USD vs. 437.15 unverified (13 kept cells) and 65.52 for the reference (peak 75.14 vs. 74.88 vs. 12.03 kW). Per-step verification restores in-bounds dispatch, but the accepted dispatch remains the LLM's repaired proposal, so the safety gain is not a KPI gain. We report this as the cost of recovery, not as a verifier-value claim, and it is the finding a longer-horizon confirmation should test.

Provenance and degradation. `first_attempt_validity` is 0.0 in every cell of both pilots *by design*: the corruptor scales every first attempt past its limit, so the metric measures the injected corruption, not agent validity; it is not comparable with the M2a screen's validity column and is not a keep-condition. Cells are kept iff `llm_backend_errors` ≤ 2 AND `llm_calls` $\geq$ steps AND telemetry is recorded; per-cell error and fallback counts for all 30 cells are in Appendix 9. Endpoint-side intermittency (429/5xx exhausting the transport's retries before fallback) is itself a reported result: 16 of 30 recovery-epoch cells took 1–3 such errors.

4.9 Service-Level Constraints: EV, Comfort, and the Failure Split

Service-level constraints are reported as implemented, not as aspirations (Table 8).

Table 8: Service-level constraints as implemented, and their reporting status.

Constraint	Definition in code	Status
EV departure charge	`service.ev_completion_rate`: fraction of departure events meeting required SOC	reported
EV margin	avg/worst realised SOC at departure	reported
Thermal comfort	hours outside [20, 25] °C indoor band	**unmeasurable** (all-NaN series)
Flexible-load completion	deferrable cycles finished in window / delay	reported
Demand response	requested / delivered / credited DR kW	reported (zero in matrix)
Cost coverage under failure	member-steps with measurable cost (scenario E)	reported
Community exchange limits	import/export kW limits	reported

Thermal comfort is unmeasurable on this dataset. The stock CityLearn 2022 dataset ships indoor dry-bulb temperature (and the other comfort series) entirely NaN, so the band check

cannot accumulate a violation and `comfort_violation_degree_hours` is 0.0 in all 250 B-axis runs, with min = max = 0.0 across every scenario-controller cell. That is a data-availability result, not comfort satisfaction: the paper makes no comfort claim, and any future comfort statement requires a dataset with real indoor temperature or a building thermal model. EV service, by contrast, is fully reported: `multi_agent` completes 0.057 of departure requirements against 0.154 for rule-based and 1.000 for MPC (Table 4).

Failure-injection results (scenario E). Across all 150 scenario-E runs (3 regimes × 5 controllers × 10 seeds) the injected failures — PV inverter, battery, EV charger at severity 1.0 (charger unavailable), sensor, and the temporary grid-service limit reduction to 95% of the import limit — produce *zero* grid-level violations: power flow converges in every step, voltage and loading stay inside limits, and no re-dispatch is triggered. Detection at the grid level is therefore empty for every failure type: removing an inverter, a battery, or a charger, blanking one building's observations, or tightening the grid-service limit does not push the community outside its limits in these runs, so no controller observes the events and detection is configuration-oracle only. Because nothing is detected, recovery is empty as well, and we make no detection or recovery claim. The member-level effect is an interruption, not a violation: one member's cost data is blanked during the failure window (`members_interrupted`), `member_cost_coverage` is 0.80 in every run, and the interruption is reported explicitly rather than scored as zero cost.

4.10 Reproducibility

Provenance record. Every run writes a `manifest.json` recording: scenario id and config SHA-256, controller and `controller_config` (including `agent_backend`, agent model digest, prompt version/hash, sampling temperature and seed), git commit and `git_dirty` flag, random seed, Python and package versions, timestep length and history steps, pricing regime, and cost basis. For LLM runs the agent block additionally records `llm_calls`, `llm_backend_errors`, `llm_latency` p50/p95, token counts, `first_attempt_validity`, `llm_unrepaired_violations`, fallback steps, repair iterations, and the hostile-proposer mode. Model identity and provider are withheld per operator policy; the digest, prompt hashes, temperature and seed in each manifest are sufficient to re-run the same proposer. `failed_runs.json` is empty wherever a matrix completed.

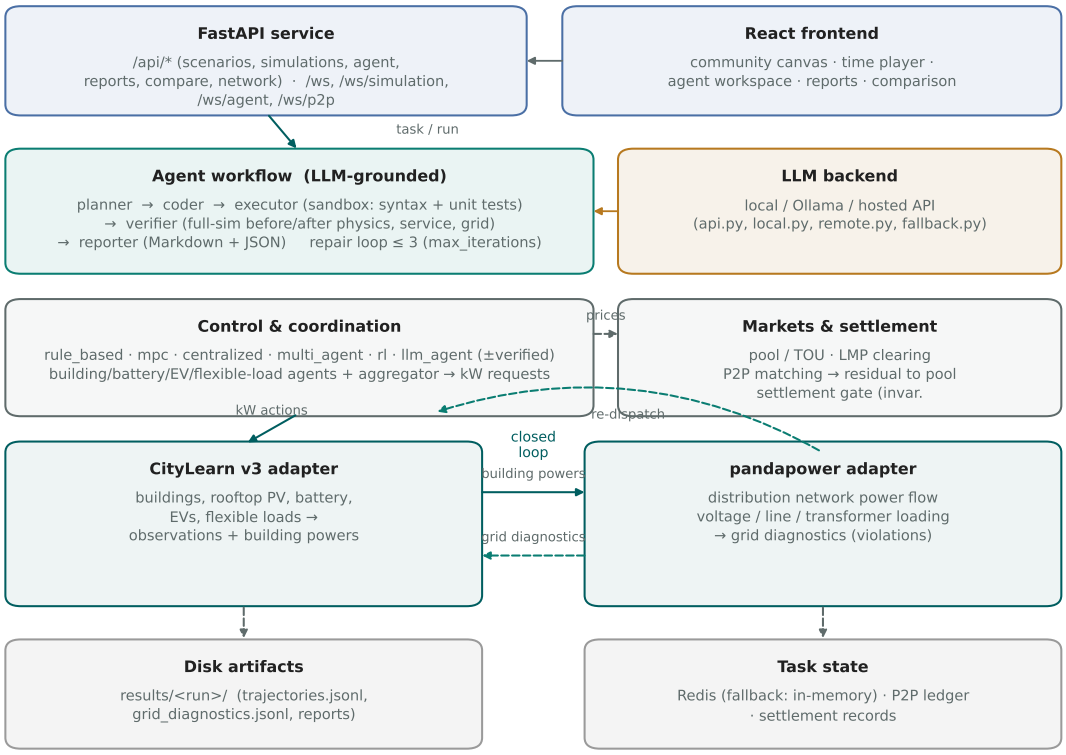
Determinism evidence. (1) *Same-seed bit-identity*: one configuration re-run three times produces byte-identical KPIs (cost 22.388099, peak 13.025067). (2) *Config distinctness*: each scenario has a distinct `scenario_config_sha256` per seed. (3) *CI tightening*: moving from 5 to 10 seeds shrinks the median 95% CI half-width by a factor of 0.539. Load/PV traces depend only on (seed, filename) (Section 3), so residual CI width reflects controller-level nondeterminism only (e.g. MPC's between-seed cost spread, ±8.54 USD in scenario A), which we state explicitly rather than present as environment variance.

Artifacts. Committed summaries live under `docs/` (D-table rebuild, M2a re-run, M2b pilots, service/failure analysis, case study, count audit); raw run directories ship as checksummed release artifacts with INDEX.json and SHA256SUMS; scenario configs, the benchmark matrix definition, and the analysis scripts are version-controlled alongside this paper's sources. Install and run instructions (dependencies, one-command smoke run, full-matrix orchestration) are maintained in the repository's `README` and `experiments/BENCHMARK_MATRIX.md`.

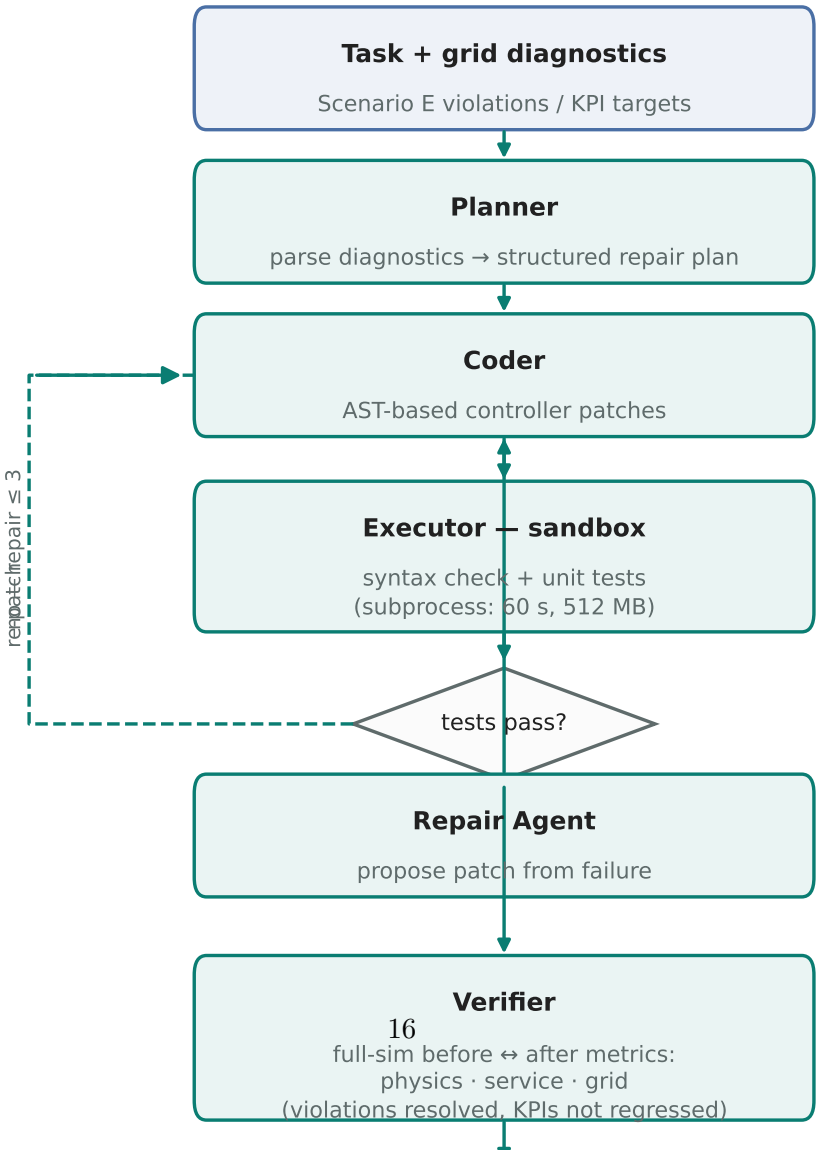
5 Discussion and Limitations

5.1 Threat Model

Adversary has full visibility of observations/actions; may inject sensor faults, grid-limit reductions, component failures. Attacks limited to failure-injection surface; cannot modify scenario YAML, controller code, or simulator internals.



(a) System architecture.



5.2 Limitations

1. **Thermal comfort is unmeasurable**: the stock dataset's indoor-temperature series is all-NaN, so `comfort_violation_degree_hours` = 0.0 everywhere and no comfort claim is made (Section 4.9).

2. **Cost basis differs by scenario**: scenario D is settlement-inclusive, A/B/C/E grid-only; all cost comparisons are within-scenario and labelled.

3. **Synthetic traces**: Load/PV profiles depend only on (seed, filename); scenario effects are device-mix effects.

4. **Controller scope**: RL is evaluated per regime with the regime-matching policy; LMP policies are trained separately.

5. **LLM proposer**: the screens use a single hosted model at temperature 0 on a 24-step horizon; model identity is withheld per operator policy, manifests retain full provenance. The M2a null and the M2b results are specific to this model–prompt–horizon combination.

6. **Horizon and seeds**: 24-step (one-day) horizons and 3-seed pilots; the pre-registered significance criterion is reserved for the full-length run.

7. **Endpoint degradation**: 16 of 30 recovery-epoch cells and one M2a cell took 1–3 transient endpoint errors; cells are quarantined under a published rule (errors ≤ 2 AND calls $\geq$ steps AND telemetry) and error/fallback shares are reported per cell (Appendix 9). A confirmation re-run of the three quarantined verified cells promoted one to keep (0 errors of 24 steps); the other two still show 3 of 24 and remain quarantined.

8. **Gini measurement gap**: the fairness coefficient does not reflect settlement redistribution; inequality claims are deferred until a settlement-aware metric exists.

9. **Weather data gap (authoring acceptance)**: no Singapore-class weather file ships with the CityLearn datasets in this environment (each dataset carries one weather.csv) and no simulator code reads `BuildingConfig.weather_file`, so the canonical weather token is the environment's honest ceiling: the weather item of the authoring corpus is reported as an environment limitation rather than passed by inventing a filename, and the authoring-acceptance coverage is 7 of 8 domains accordingly.

10. **Failure-injection coverage**: the reported scenario-E runs use the paper's aggressive failure configuration (EV charger severity 1.0, grid-service limit reduced to 95% of the import limit, identical in all three regime legs; A/B/C/D use the standard configuration). No injected failure produces a grid-level violation in any of the 150 runs, so detection and recovery are reported as empty rather than claimed.

5.3 Case Study: Verified Repair Under a Corrupted Proposer

The clearest single-cell view of the machinery is one cell of the recovery pilot (scenario A, seed 42, 24 steps, pool TOU, hosted model). The planner packages the observation (load, PV, battery SOC, EV arrival/departure, price) into the versioned dispatch prompt; the coder emits a JSON dispatch proposal; the executor applies it; the verifier checks asset shape and device power bounds *before* the step. In this cell, 23 of 24 first attempts on the verified arm were corrupted past their limits (one call failed at the endpoint and took the deterministic replacement); the bounds check fired 23 times, the corrective re-prompt returned an in-bounds proposal that was applied (recorded repair success 0.957, one deterministic replacement), and the reporter wrote Markdown + JSON. The unverified arm of the same cell applied all corrupted proposals

as proposed (adapter clamping downstream) at 460.87 USD against the verified arm's 471.15; the deterministic reference for that same (scenario, seed) cell — `scenario_a_multi_agent_42`, one of the 15 reference cells of Table 7 — is 66.29 USD. The safety gain is therefore real and measurable at the proposal layer; the KPI gain is not (the verified arm is slightly *worse* on cost here), which is the honest reading of the recovery pilot at cell granularity.

6 Conclusion

We present a service-aware energy agent framework spanning pricing regimes, failure injection, and verified LLM dispatch, evaluated on 750 pricing-regime runs plus a 150-run scenario-D rerun, a 150-cell real-LLM screen, and 60 hostile-proposer cells. The multi-agent controller is cost- and peak-minimal in every scenario under every regime with an explicit EV trade-off that we report rather than resolve; only price-aware controllers change dispatch across pricing regimes, while cost remains a function of the billed price series on any fixed trajectory; P2P clearing redistributes among members without aggregate benefit for A/B/C/E while scenario D diverges through the real grid-cost change from energy sharing under a verified settlement accounting invariant. Under the injected failure configuration, no failure type produces a grid-level violation in any run, so detection and recovery are reported as empty rather than claimed. The verification machinery is a no-op under an in-bounds proposer (a proposal-layer safety-metrics null that we report as such), and under a deliberately out-of-bounds proposer it accepts zero corrupted proposals and repairs 0.909–1.000 of them (13 of 15 verified cells kept), at the measured cost that the accepted dispatch remains the LLM's repaired proposal; the clamp the simulation adapter applies is shared by both arms, so these are proposal-layer results, not physical-safety results. The five-agent code-repair loop is implemented and instrumented but not yet demonstrated to change behaviour; a code-repair benchmark is future work. Thermal comfort is unmeasurable on this dataset and no comfort claim is made. All artifacts are reproducible from manifests, committed summaries, and checksummed release artifacts.

Future work: a longer-horizon, multi-seed confirmation of the recovery result; a settlement-aware fairness metric for redistribution analysis; a dataset with real indoor temperature for comfort claims; a deterministic (non-LLM) bounds-check baseline to price the LLM verification layer itself; and a code-repair benchmark against seeded controller defects.

A Per-Cell Degradation Detail (M2b)

The M2a screen quarantines one cell on the same rule: `scenario_d_llm_agent_verified_1234` (1 of 24 steps backend-error-degraded, first-attempt validity 0.958), left out of the 99 usable LLM-arm runs and reported here for completeness.

References

Fraol Batole, Foutse Khomh, and Hridesh Rajan. Refineact: Automatic runtime verification of llm agent actions. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2026.

Reda El Makroum, Sebastian Zwickl-Bernhard, and Lukas Kranzl. Agentic ai home energy management system: A large language model framework for residential load scheduling. *Results in Engineering*, 29:109857, 2026.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

Table 9: Per-cell endpoint-degradation detail for the M2b recovery epoch (30 cells; the keep rule is `llm_backend_errors` ≤ 2 AND `llm_calls` $\geq$ steps AND telemetry recorded). Quarantined cells are excluded from headline statistics; their per-cell values are reported here. Raw L1 counts (out-of-bounds proposal values forwarded by the unverified arm; blocked at the proposal layer in the verified arm) are in `docs/reviewer_response/e1_cells.csv`.

Scenario	Seed	Arm	LLM calls	Backend errors	Fallback steps	Repair success	Keep
a	123	unverified	24	1	1	–	KEEP
a	123	verified	47	1	1	0.9565	KEEP
a	42	unverified	24	2	2	–	KEEP
a	42	verified	47	1	1	0.9565	KEEP
a	456	unverified	24	1	1	–	KEEP
a	456	verified	48	0	0	1.0	KEEP
b	123	unverified	24	0	0	–	KEEP
b	123	verified	48	0	0	1.0	KEEP
b	42	unverified	24	2	2	–	KEEP
b	42	verified	48	0	0	1.0	KEEP
b	456	unverified	24	2	2	–	KEEP
b	456	verified	47	2	2	0.9166	KEEP
c	123	unverified	24	0	0	–	KEEP
c	123	verified	47	3	3	0.88	quarantine
c	42	unverified	24	0	0	–	KEEP
c	42	verified	48	0	0	1.0	KEEP
c	456	unverified	24	0	0	–	KEEP
c	456	verified	46	2	2	0.9090	KEEP
d	123	unverified	24	0	0	–	KEEP
d	123	verified	47	1	1	0.9565	KEEP
d	42	unverified	24	1	1	–	KEEP
d	42	verified	48	0	0	1.0	KEEP
d	456	unverified	24	0	0	–	KEEP
d	456	verified	48	0	0	1.0	KEEP
e	123	unverified	24	0	0	–	KEEP
e	123	verified	48	0	0	1.0	KEEP
e	42	unverified	24	0	0	–	KEEP
e	42	verified	48	3	3	0.8888	quarantine
e	456	unverified	24	1	2	–	KEEP
e	456	verified	47	2	2	0.9166	KEEP

Yusuf Izmirlioglu, Loc Pham, Tran Cao Son, and Enrico Pontelli. A survey of multi-agent systems for smartgrids. *Energies*, 17(15):3620, 2024. doi: 10.3390/en17153620.

Sokipriala Jonah, Queen Moses, Abiola Babatunde, Michael Ajao-Olarinoye, and Daniel Bammeke. Llms for agentic home energy management. *arXiv:2607.04569*, 2026.

Wooyoung Jung. Multi-agent home energy management assistant (hema). *SoftwareX*, 34:102633, 2026.

Shanxiang Mao, Muyang Liu, Junru Chen, Dongliang Nan, and Jian Ma. A trustworthy intent-driven llm-agent workflow with state verification for power system simulation. *Applied Energy*, 427:128868, 2026. doi: 10.1016/j.apenergy.2026.128868.

Costas Mylonas, Magda Foti, Andrea Pomarico, Matheus Duarte, Qian Zhang, and Emmanouel Varvarigos. Poweragentbench-ss: A benchmark for agentic ai in power system steady-state studies. *arXiv:2606.18789*, 2026.

Maik Rall, Paul Boock, Christoph Heinze, and Bernhard Kuchen. Program repair using abstract syntax trees and machine learning. In *IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2020.

James B. Rawlings, David Q. Mayne, and Moritz Diehl. Model predictive control: Theory, computation, and design. *Nob Hill Publishing*, 2020.

Daniela Rojas, Abdulwahab Albassam, Aidan G. Leung, Jett Ngo, Ryan Luo, Peter R. Quawas, Junpyung Kim, Kangkai Liang, Mansi Nanavati, Jonathan Mai, Meng-Chi Tsai, Yun-Tong Tsai, Yize Chen, and Yuanyuan Shi. Llms and agentic ai systems for smart grids: A tutorial on architectures and applications. *arXiv:2607.18147*, 2026.

Yifan Su, Zhaojian Wang, Ming Cao, Mengshuo Jia, and Feng Liu. Convergence analysis of dual decomposition algorithm in distributed optimization: Asynchrony and inexactness. *arXiv:2103.02784*, 2021.

Leon Thurner, Alexander Scheidler, Florian Schäfer, Jan-Hendrik Menke, Julian Dollichon, Frieder Meier, Steffen Meinecke, and Martin Braun. pandapower — an open-source python tool for convenient modeling, analysis, and optimization of electric power systems. In *IEEE Transactions on Power Systems*, volume 33, pages 6510–6521, 2018. doi: 10.1109/TPWRS.2018.2829021.

José R. Vázquez-Canteli, Jérôme Kämpf, Gregor Henze, and Zoltán Nagy. Citylearn v1.0: An openai gym environment for demand response with deep reinforcement learning. *Proceedings of the 6th ACM International Conference on Systems for Energy-Efficient Built Areas (BuildSys)*, pages 176–179, 2020. doi: 10.1145/3408308.3427976.

Vassilios Yfantis, Achim Wagner, and Martin Ruskowski. Numerical benchmarking of dual decomposition-based optimization algorithms for distributed model predictive control. *Results in Control and Optimization*, 17:1–19, 2024.

Xiyuan Zhou, Huan Zhao, Yuheng Cheng, Gaoqi Liang, Guolong Liu, Wenxuan Liu, Yan Xu, and Junhua Zhao. Elecbench: A power grid dispatch evaluation benchmark for large language models. In *2025 IEEE Power & Energy Society General Meeting (PESGM)*, 2025. doi: 10.1109/PESGM52009.2025.11225826.

Zhi Zhou, Fang Zhao, Jie Wang, and Yan Liu. Deep reinforcement learning for demand response in microgrids: A review. In *ACM e-Energy*, 2022.